ACM ICPC 铜牌算法板子 完整版

手敲时按说明一并抄入依赖模块；合并进同一 .cpp 后省略本地头文件 #include 与 #pragma once，保留 bits/stdc++.h。cp:: 按调用示例使用。

先识别题型、写出证明，再按稳定编号查模板。所有正式代码都由仓库源码自动生成。

赛站纸质资料数量与规格：待赛站确认。25 页不是已核实的成都站限制。

统一提醒：多测新建容器；类型与区间以各模块契约为准；所有算术结果须处于声明范围。

思维卡片见第 2–3 页；排错与最后复习见第 4 页。

编号	算法与识别入口	页码
CO01	主程序骨架 Main	5
CO02	前缀、差分、离散化 Basics	6
CO03	答案二分、双指针、窗口 Search	8
DP01	0/1 与完全背包 Knapsack	9
DP02	线性 DP 与最长递增子序列 LIS	10
DP03	小规模状压最短路径 Hamilton DP	11
DS01	树状数组 Fenwick	12
DS02	单点线段树 Segment Tree	13
DS03	懒标记线段树 Lazy Segment Tree	14
DS04	单调栈与单调队列 Monotonic	16
G01	并查集 Disjoint Set Union	17
G02	BFS 最短路与路径恢复 BFS Path	18
G03	迭代 DFS 连通块与父亲 Iterative DFS Forest	19
G04	拓扑排序 Topological Sort	20
G05	简单无向基环图环恢复 Unicyclic Graph	21
M01	整数与位运算 Integer and Bits	23
M02	安全模运算与快速幂 Modular Power	24
DP04	石子合并区间 DP Interval Merge	25
DS05	稀疏表 Sparse Table	26
G06	单源与多源最短路 Dijkstra	27
G07	Kruskal 最小生成树与森林 Kruskal MST	29
G08	堆优化 Prim 最小生成森林 Prim MST	30
G09	树上倍增最近公共祖先 Binary Lifting LCA	31
G10	Euler 子树区间与迭代树 DP Euler Subtree	33
M03	普通模矩阵快速幂 Matrix Power	34
M04	乘积递推修正版 Product Recurrence	35
M05	质数模数组合数 Prime Combinations	36
S01	KMP 与 Z 函数 Prefix Matching	37
S02	双模字符串哈希 Double Hash	38
S03	基础字符串处理 String Basics	39
G11	强连通分量 Iterative Kosaraju SCC	40
G12	无向图找桥 Iterative Bridges	41

思维卡片 从题意走到证明

构造 先证必要 再证足够

先枚举 n≤6 的小数据，记录可行性；从总和、奇偶、极值寻找必要条件。随后给出达到条件的构造，并逐条验证题目限制。找到一种答案不等于证明所有可行情形。

例：cf/construction/1.cpp 的回文构造；先传播固定镜像，再扣计数与处理中点，最后独立检查长度、数量和回文。代码记录不代表 AC。

操作 每一步改变了什么

把一次操作写成变量变化式。检查总和、差值、奇偶、模数、连通性。先证明操作次数下界，再考虑能否达到；不变量相同只是必要条件，可能仍不可达。

例：2025 成都 D，状态可重复出现，要记录访问状态和前驱；搜索终点后沿前驱恢复合法操作。

贪心 先找反例 再找交换

写清局部选择，枚举小数据寻找反例。尝试把某个最优方案的第一步换成自己的选择，证明可行且答案不变差。无法完成交换时不要把直觉当证明。

例：2025 成都 J 的分组调度与 C 的逆向排序；练习把剩余问题的结构写在纸上。

模拟 状态与更新次序

列出会变化的全部状态。按题面一次操作的真实先后更新，区分同时更新与依次更新；画出第 0 步、第一次、最后一次。先保证直接模拟正确，再证明跳步。

例：2024 成都 J 与 2025 成都 J，优先读清事件顺序，不因题短而省掉手算。

思维卡片 从暴力走到算法

区间操作 试着写差分

对闭区间 [l,r] 加 x，差分只改 d[l]+=x、d[r+1]-=x。观察正负差分如何配对。最优操作数必须有下界证明，不能把树状数组或线段树当成建模本身。

现场：画出 n=1、l=r、r=n 三个例子，明确是否需要 n+2 个位置。

图论 先辨认图的结构

先问：有向还是无向？是否连通？是否允许重边或自环？无向森林边数为 n-连通块数。简单连通基环图满足 m=n；去叶后得到环，再沿环恢复顺序。

现场：DFS 必须区分父边与已访问节点；路径恢复用前驱，不把遍历顺序当路径。

DP 保留影响未来的信息

从暴力决策树开始。两段历史若对所有未来选择的影响一样，就能合并成同一状态。先写状态含义、初值、合法转移和答案位置，再考虑滚动数组。

例：0/1 背包处理第 i 件物品；倒序容量保证每件只用一次。完全背包正序容量允许同一件继续使用。

复杂度 先用约束排除

n≈20 才考虑 2^n 状态；n≈2e5 通常需要近线性或 n log n。数值大不等于状态多，寻找周期、矩阵或二分前必须证明能跳过的过程。

例：2025 成都 B 是 max-plus 状态转移矩阵，不能直接套普通模数乘法矩阵。

现场排错与最后复习

交题前 90 秒

手算最小规模、全部相同、严格递增/递减、最大值、无解、重复值。多测至少连跑两组不同大小。核对输入格式、初值、输出换行、数组容量与时间复杂度。

读题时把区间约定写在代码第一行。此书数组与图优先 1-indexed 闭区间，字符串/状态等例外逐模块注明。

WA

对照变量的数学含义逐行检查。重点查 < 与 ≤、前缀 sum(l,r)=sum(r)-sum(l-1)、懒标记下传、负数取模、不可达状态参与转移、相等元素处理、排序比较器严格弱序。

先保存失败输入，再做小数据枚举与独立暴力；不要同时改算法和多个边界。

RE 与整数溢出

越界检查空容器、1-indexed 的 n+1、差分的 r+1；避免深树递归。乘法先提升：1LL*a*b 或 (__int128)a*b。INF 加边权前检查不可达，long long 也有上限。

__builtin_ctz(0) 与 __builtin_clz(0) 未定义；位移使用 1ULL，位数必须小于类型位宽。

TLE 与止损

区分死循环、重复入队和复杂度超限。二分每轮必须缩小区间；Dijkstra 忽略过期堆条目。一次调试围绕一个假设，15 分钟无新证据就换人复核或切题。

五小时：前 20 分钟并行读题与找稳题；中段持续汇总候选题与证明；最后 40 分钟优先已证思路、短修复和现有提交。按实际熟悉程度分工，不预设谁擅长什么。

赛前最后一轮

先读思维卡片和本页；手敲并查集、Fenwick、Lazy 调用各一遍；复盘一题自己的构造和图论失误；确认纸质资料规则、工具链和睡眠。不要在最后一晚从零学习重型算法。

板子帮助减少编码错误，不能替代性质、证明与团队时间分配，也不能保证获铜牌。

CO01 主程序骨架 Main

识别：每场比赛新建文件；单测、多测都从这份骨架开始。

约定：GNU++20；默认单测，只有题面给出测试组数时才取消 cin >> T 注释。

复杂度：骨架 O(T) 次调用；具体复杂度由 solve 决定。

调用：把题目逻辑写进 solve()；需要多测时启用 cin >> T。

易错：把单测题第一个数字误读为 T。；关闭同步后不要混用 scanf 与 cin。；多测数据、答案和容器应在 solve 内初始化。

例题：在 solve 内先读 n，再声明 vector<ll> a(n+1)并读入 a[1..n]。仓库实例：cf/date/7/7.25/1.cpp，main 读取 T 并逐组调用 solve。

源码 core/main.cpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// GNU++20；数组常用 vector<ll> a(n + 1)，有效范围 [1,n]。
// 和、乘积可能超过 ll 时，先转 __int128 再运算。
void solve() {
    // 在这里读入并处理一组数据；每组状态尽量使用局部变量。
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // 多测题取消下一行注释；单测题保持 T=1。
    // cin >> T;
    while (T--) solve();
    return 0;
}
```

CO02 前缀、差分、离散化 Basics

识别：静态区间和用前缀；离线区间加用差分；值域太大先离散化。

约定：a[1..n]，{}或{0}为空；非空查询须 1<=l<=r<=n，l>r 为 0 或不修改。Difference 初值全 0；所有 ll 中间值须不溢出。rank 只查已存在值。

复杂度：前缀建表 O(n)、查询 O(1)；差分修改 O(1)、还原 O(n)；离散化 O(n log n)、rank O(log n)；空间 O(n)。

调用：PrefixSum p(a); p.sum(l,r); Difference d(n); d.add(l,r,x); auto b=d.values(); Compression c(a); c.rank(a[i]); cout<<to_string_i128(x);

易错：前缀差是 pre[r]-pre[l-1]。；离散化保顺序，不保原距离；values[rank]还原原值。；__int128 最小值不能直接取负；输出前必须转字符串。

例题：a={0,4,-2,4}：PrefixSum(a).sum(2,3)=2；Compression(a).rank(4)=2。实题：洛谷 P8218 求区间和（静态闭区间求和）：洛谷 P8218 。

源码 core/basics.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 数组 a[1..n]；{} 或 {0} 表示 n=0。所有 ll 中间结果须不溢出。
inline string to_string_i128(__int128 x) {
    bool neg = x < 0;
    // 无符号取反能处理 -2^127，禁止直接对最小有符号数取负。
    __uint128_t u = neg ? -(__uint128_t)x : (__uint128_t)x;
    string s;
    do { s += char('0' + u % 10); u /= 10; } while (u);
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
struct PrefixSum {
    int n;
    vector<ll> pre;
    explicit PrefixSum(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), pre(n + 1) {
        for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
    }
    ll sum(int l, int r) const {
        if (l > r) return 0; // 空区间；非空时 1<=l<=r<=n。
        assert(1 <= l && r <= n);
        return pre[r] - pre[l - 1];
    }
};
// 离线区间加；初值全 0。values() 得到 1 下标数组，可重复调用。
struct Difference {
    int n;
    vector<ll> d;
    explicit Difference(int size = 0) : n(size), d(size + 1) {
        assert(size >= 0);
    }
    void add(int l, int r, ll x) {
        if (l > r) return;
        assert(1 <= l && r <= n);
        d[l] += x;
        if (r < n) d[r + 1] -= x;
    }
    vector<ll> values() const {
        vector<ll> a(n + 1);
        for (int i = 1; i <= n; ++i) a[i] = a[i - 1] + d[i];
        return a;
    }
};
```

CO02 前缀、差分、离散化 Basics 续 2

接上一页；请合并复制整个模块。源码 core/basics.hpp

```
// 离散化保序不保距离。rank(x) 只接收初始化时存在的值。
struct Compression {
    vector<ll> values; // values[rank] 为原值；values[0] 不使用。
    explicit Compression(const vector<ll>& a) {
        values.assign(a.begin() + min<size_t>(1, a.size()), a.end());
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        values.insert(values.begin(), 0);
    }
    int rank(ll x) const {
        auto it = lower_bound(values.begin() + 1, values.end(), x);
        assert(it != values.end() && *it == x);
        return int(it - values.begin());
    }
};
```

CO03 答案二分、双指针、窗口 Search

识别：答案满足单调判断时二分；有序数组找二元和用双指针；非负数组求最长受限和区间用窗口。

约定：二分范围为整数闭区间；first_true 要求假到真，last_true 要求真到假，无解/空区间为 nullopt。数组 1 下标；two_sum 需非降序，窗口元素须非负。

复杂度：二分 O(log 范围)次 check、O(1)空间；双指针和窗口 O(n)时间、O(1)额外空间。

调用：auto ans=first_true(lo,hi,check); if(ans) cout<<*ans; auto [i,j]=two_sum_sorted(a,target); int len=longest_sum_at_most(a,k);

易错：必须先证明 check 单调；check 自己的乘加仍可能溢出。；two_sum 返回不同下标 i<j；无解为{0,0}。；窗口含负数时不能套用；limit<0 返回 0。

例题：first_true(1,10,[](ll x){return x>=7;})得到 7；窗口{0,2,1,3}、limit=3 的最长长度为 2。仓库实例：cf/other/Hello_2018/D.cpp，按时间排序后 check(x)判断能否选 x 题，二分最大可行 x。

源码 core/search_window.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 整数闭区间 [lo,hi]；空区间或无解返回 nullopt。
// ok 确定且单调；check 内部的乘加仍需自行防溢出。
template<class F>
optional<ll> first_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(hi)) return nullopt; // false...false,true...true
    while (lo < hi) {
        ll mid = midpoint(lo, hi); // 向 lo 取整，全 ll 范围不溢出。
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
template<class F>
optional<ll> last_true(ll lo, ll hi, F ok) {
    if (lo > hi || !ok(lo)) return nullopt; // true...true,false...false
    while (lo < hi) {
        ll mid = midpoint(hi, lo); // 向 hi 取整，避免死循环。
        if (ok(mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
// a[1..n] 非降序，找不同位置 i<j 且 a[i]+a[j]=target；无解 {0,0}。
inline pair<int, int> two_sum_sorted(const vector<ll>& a, ll target) {
    int l = 1, r = max(0, int(a.size()) - 1);
    while (l < r) {
        __int128 s = (__int128)a[l] + a[r];
        if (s == target) return {l, r};
        if (s < target) ++l;
        else --r;
    }
    return {0, 0};
}
// a[1..n] 必须非负；返回和<=limit 的最长连续段长度，可为 0。
// 负数会破坏窗口单调性；{}、{0} 均返回 0。和用 i128 防溢出。
inline int longest_sum_at_most(const vector<ll>& a, ll limit) {
    if (limit < 0) return 0;
    int n = max(0, int(a.size()) - 1), l = 1, best = 0;
    __int128 sum = 0;
    for (int r = 1; r <= n; ++r) {
        assert(a[r] >= 0);
        sum += a[r];
        while (sum > limit) sum -= a[l++];
        best = max(best, r - l + 1);
    }
    return best;
}
```

DP01 0/1 与完全背包 Knapsack

识别：按物品选择、总重量受容量限制，价值最大化；能区分每件一次与无限次。

约定：w、v 从 1 开始，w[i]>0，cap>=0；价值和任意可达总价值绝对值<2^60。默认总重量不超过容量；exact=true 表示恰好重量，不可达为 KNAP_NEG。

复杂度：时间 O(n*cap)，空间 O(cap)。

调用：vector<int> w(n+1); vector<ll> v(n+1); 读入 1..n；auto f=knapsack01(w,v,m,true); 判断 f[m]==KNAP_NEG 后输出。无限次改调 knapsack_complete。

易错：0/1 容量倒序，完全背包容量正序。；恰好装满必须只初始化 dp[0]=0，其他为负无穷；不能把负无穷直接相加。；不支持零重量物品；完全背包零重正价值会使答案无界。

例题：原仓库 01bag.cpp：https://github.com/ball-castle/template/blob/main/01bag.cpp 。保留原正确转移，新增 64 位与恰好容量契约；不宣称在线 AC。

源码 dp/knapsack.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w、v 为 1-indexed；w[i]>0，cap>=0，价值及可达总和绝对值<2^60。
// exact=false：总重量<=j；exact=true：总重量恰好为 j。
// 不可达为 KNAP_NEG；不能直接对不可达状态加价值。
inline constexpr ll KNAP_NEG = -(1LL << 60);
inline vector<ll> knapsack01(const vector<int>& w,
                             const vector<ll>& v, int cap,
                             bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 倒序：dp[j-w[i]] 仍属于上一轮，物品最多取一次。
        for (int j = cap; j >= w[i]; --j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}

inline vector<ll> knapsack_complete(const vector<int>& w,
                                    const vector<ll>& v, int cap,
                                    bool exact = false) {
    assert(cap >= 0 && !w.empty() && w.size() == v.size());
    vector<ll> dp(cap + 1, exact ? KNAP_NEG : 0);
    dp[0] = 0;
    for (int i = 1; i < int(w.size()); ++i) {
        assert(w[i] > 0);
        // 正序：dp[j-w[i]] 可以已经取过本物品，允许继续取。
        for (int j = w[i]; j <= cap; ++j)
            if (dp[j - w[i]] != KNAP_NEG)
                dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp;
}
// auto best = knapsack01(w, v, capacity, true);
// best[capacity]==KNAP_NEG 表示不能恰好装满；否则就是最大价值。
```

DP02 线性 DP 与最长递增子序列 LIS

识别：寻找保持原顺序的最长严格递增或非递减子序列；先用朴素状态理解，再用二分优化。

约定：a 从 1 开始，空序列传 {0}；strict=true 严格递增，false 非递减。只返回长度，不恢复实际序列。

复杂度：lis_quadratic：O(n^2) 时间；lis_length：O(n log n) 时间；空间均 O(n)。

调用：vector<ll> a(n+1); 读入 a[1..n]；int ans=lis_length(a); 初学调试可用 lis_quadratic(a)对照。

易错：严格递增对应 lower_bound；重复值可接在后面时用 upper_bound。；tails 表示各长度的最小末尾，最终 tails 通常不是原数组的一个子序列。

例题：原仓库线性 dp.cpp：https://github.com/ball-castle/template/blob/main/%E7%BA%BF%E6%80%A7dp.cpp 。保留原 O(n^2) 状态，增加二分版。

源码 dp/lis.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a 为 1-indexed；仅求长度。strict=true 严格递增，否则非递减。
// 最直观状态：dp[i] 为以 a[i] 结尾的最优长度，O(n^2)。
inline int lis_quadratic(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    int n = int(a.size()) - 1, ans = 0;
    vector<int> dp(n + 1, 1);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j < i; ++j) {
            bool can = strict ? a[j] < a[i] : a[j] <= a[i];
            if (can) dp[i] = max(dp[i], dp[j] + 1);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}

// tails[k]：长度 k+1 的序列中，最小的末尾值；它不是答案序列。
// 严格递增用 lower_bound，非递减用 upper_bound；O(n log n)。
inline int lis_length(const vector<ll>& a, bool strict = true) {
    assert(!a.empty());
    vector<ll> tails;
    for (int i = 1; i < int(a.size()); ++i) {
        auto it = strict ? lower_bound(tails.begin(), tails.end(), a[i])
                         : upper_bound(tails.begin(), tails.end(), a[i]);
        if (it == tails.end()) tails.push_back(a[i]);
        else *it = a[i];
    }
    return int(tails.size());
}
// vector<ll> a(n+1); 读入 a[1..n]; cout << lis_length(a);
```

DP03 小规模状压最短路径 Hamilton DP

识别：n 很小，需要每个点恰好访问一次；用 mask 记录已选集合，末尾点作为第二维。

约定：w 为 1-indexed 有向图方阵，缺边为 HAMILTON_INF；从 1 到 n，每点恰好一次。0<=n<=18，n=0/1 约定为 0；允许负边，任意路径总权值在(-2^60,2^60)。

复杂度：时间 O(n^2*2^n)，空间 O(n*2^n)；n=18 约 36MiB。

调用：vector<vector<ll>> w(n+1,vector<ll>(n+1,HAMILTON_INF)); 填边；ll ans=hamilton_path(w); ans==HAMILTON_INF 表示无解。

易错：点 i 对应第 i-1 位，状态里的 u 是 0-indexed。；最短路、方案计数、是否可行的转移不同，不能原封不动混用。；终点固定为 n；任意终点需对最终 mask 的末尾状态取最小值。

例题：相关变体 CSES Hamiltonian Flights：https://cses.fi/problemset/task/1690 。原题计数且 n<=20，需把 min 转移改成求和并重新评估复杂度；当前模板是最短路，未在线提交。

源码 dp/hamilton_path.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// w 为 1-indexed 方阵；有向图，缺边用 HAMILTON_INF。
// 从 1 出发，到 n 结束，每个点恰好一次。n<=18；n=0/1 约定答案 0。
// 可以有负边；任意路径的权值总和须在 (-2^60,2^60) 内。
inline constexpr ll HAMILTON_INF = 1LL << 60;
inline ll hamilton_path(const vector<vector<ll>>& w) {
    assert(!w.empty());
    int n = int(w.size()) - 1;
    assert(n <= 18);
    for (const auto& row : w) assert(row.size() == w.size());
    if (n <= 1) return 0;
    int states = 1 << n;
    // 点 i 对应第 i-1 位；dp[mask*n+u] 中 u 是 0-indexed。
    // 状态表示已经访问 mask、当前停在 u+1 的最短路。
    vector<ll> dp(size_t(states) * n, HAMILTON_INF);
    dp[n] = 0;  // mask=1，停在点 1。
    for (int mask = 1; mask < states; mask += 2) {
        for (int u = 0; u < n; ++u) {
            ll cur = dp[size_t(mask) * n + u];
            if (cur == HAMILTON_INF) continue;
            for (int v = 0; v < n; ++v) {
                if ((mask >> v & 1) || w[u + 1][v + 1] == HAMILTON_INF)
                    continue;
                int next = mask | (1 << v);
                ll& dst = dp[size_t(next) * n + v];
                dst = min(dst, cur + w[u + 1][v + 1]);
            }
        }
    }
    return dp[size_t(states - 1) * n + n - 1];
}
// O(n^2*2^n) 时间、O(n*2^n) 空间；n=18 的 dp 约 36MiB。
// 返回 HAMILTON_INF 表示不存在；求任意终点则改取最后一行最小值。
```

DS01 树状数组 Fenwick

识别：单点加、前缀和、区间和；频次累计、逆序对可先离散化。

约定：n>=0，1 下标；add 位置须 1..n，sum 允许 0；range_sum(l,r)的 l>r 返回 0。树节点、前缀、答案及中间值均须在 ll 范围。

复杂度：单次增量/查询 O(log n)，空间 O(n)；逐点初始化 O(n log n)。

调用：Fenwick f(n); for(int i=1;i<=n;++i) f.add(i,a[i]); f.add(p,delta); ll ans=f.range_sum(l,r);

易错：add(0,x)非法，旧写法会死循环。；这里是增量 add；改成新值要传 new-old。；区间最小值不能照搬两个前缀相减。

例题：由旧 Fenwick_tree.cpp 提炼；修复 rangeSum 与 query 接口不一致。实题：洛谷 P3374 树状数组 1，单点加与闭区间求和：洛谷 P3374 。

源码 data_structure/fenwick.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 单点加、区间和。n>=0，1 下标；所有树节点/前缀/答案须在 ll 范围。
// 适用于可相减的前缀信息，不能直接把求和改成区间最小值。
struct Fenwick {
    int n;
    vector<ll> bit;
    explicit Fenwick(int size = 0) : n(size), bit(size + 1) {
        assert(size >= 0);
    }
    void add(int p, ll x) {
        assert(1 <= p && p <= n); // p=0 会使 lowbit 循环无法前进。
        while (p <= n) {
            bit[p] += x;
            int step = p & -p;
            if (step > n - p) break; // 避免索引加法越过 int 上界。
            p += step;
        }
    }
    ll sum(int p) const { // [1,p]，允许 p=0。
        assert(0 <= p && p <= n);
        ll ans = 0;
        for (; p; p -= p & -p) ans += bit[p];
        return ans;
    }
    ll range_sum(int l, int r) const { // [l,r]；空区间返回 0。
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return sum(r) - sum(l - 1);
    }
};
```

DS02 单点线段树 Segment Tree

识别：单点赋值、区间求和；需要以后扩展节点信息时使用。

约定：a[1..n]；{}或{0}为空树；set 位置须 1..n，非空 sum 须 1<=l<=r<=n，空区间返回 0。n<=INT_MAX/4；区间和及中间值须在 ll 范围。

复杂度：建树 O(n)；赋值/查询 O(log n)；空间 O(n)。

调用：SegmentTree seg(a); seg.set(p,x); ll ans=seg.sum(l,r);

易错：set(p,x)是赋值，不是加 x。；修改返回时必须 pull；答案与节点都用 ll。；空树只允许空查询，不允许 set。

例题：保留旧 segmen tree1.cpp 的递归建树/合并方法。仓库实例：cf/segment_tree_cf_edu/1.cpp，单点赋值、区间和；输入 0 下标[l,r)转成内部 1 下标[l+1,r]。

源码 data_structure/segment_tree.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 单点赋值、区间求和；a[1..n]，{} 或 {0} 表示空树。
// 任意区间和及中间值须在 ll 范围；n<=INT_MAX/4 且内存足够。
struct SegmentTree {
    int n;
    vector<ll> tr;
    explicit SegmentTree(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), tr(4ULL * max(1, n)) {
        if (n) build(1, 1, n, a);
    }
    void set(int pos, ll x) { // a[pos]=x；不是单点加。
        assert(1 <= pos && pos <= n);
        update(1, 1, n, pos, x);
    }
    ll sum(int l, int r) const {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(1, 1, n, l, r);
    }
private:
    void pull(int p) { tr[p] = tr[p * 2] + tr[p * 2 + 1]; }
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) { tr[p] = a[l]; return; }
        int m = l + (r - l) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }
    void update(int p, int l, int r, int pos, ll x) {
        if (l == r) { tr[p] = x; return; }
        int m = l + (r - l) / 2;
        if (pos <= m) update(p * 2, l, m, pos, x);
        else update(p * 2 + 1, m + 1, r, pos, x);
        pull(p);
    }
    ll query(int p, int l, int r, int L, int R) const {
        if (L <= l && r <= R) return tr[p];
        int m = l + (r - l) / 2;
        ll ans = 0;
        if (L <= m) ans += query(p * 2, l, m, L, R);
        if (R > m) ans += query(p * 2 + 1, m + 1, r, L, R);
        return ans;
    }
};
```

DS03 懒标记线段树 Lazy Segment Tree

识别：在线区间加、区间和；更新整段后仍需查询任意子段。

约定：a[1..n]；{}或{0}为空树；l>r 时 add 无操作、sum 为 0，非空范围须合法。n<=INT_MAX/4；节点和、累积 tag、长度*x 及中间值均须在 ll 范围。

复杂度：建树 O(n)；区间加/求和 O(log n)；空间 O(n)。

调用：LazySegmentTree seg(a); seg.add(l,r,delta); ll ans=seg.sum(l,r);

易错：tag 表示已计入本节点、尚未传给子节点的增量。；查询部分覆盖时也必须 push；完全覆盖直接返回。；本模板仅有加法标记，不可直接混入赋值或乘法。

例题：旧 Segment tree2.cpp 最小反例：[0,0]全段+1 后查[1,1]，旧值 0、正确值 1。实题：洛谷 P3372 线段树 1，区间加与区间和：洛谷 P3372 。

源码 data_structure/lazy_segment_tree.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 区间加、区间和；a[1..n]，{} 或 {0} 为空；n<=INT_MAX/4。
// 节点和、标记累加、长度*x 及所有中间值必须在 ll 范围。
struct LazySegmentTree {
    int n;
    vector<ll> tr, tag;
    explicit LazySegmentTree(const vector<ll>& a)
        : n(max(0, int(a.size()) - 1)), tr(4ULL * max(1, n)), tag(tr.size()) {
        if (n) build(1, 1, n, a);
    }
    void add(int l, int r, ll x) {
        if (l > r) return; // 空区间不修改，包括 n=0。
        assert(1 <= l && r <= n);
        modify(1, 1, n, l, r, x);
    }
    ll sum(int l, int r) {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(1, 1, n, l, r);
    }
private:
    void pull(int p) { tr[p] = tr[p * 2] + tr[p * 2 + 1]; }
    void apply(int p, int l, int r, ll x) {
        tr[p] += (r - l + 1LL) * x;
        tag[p] += x;
    }
    void push(int p, int l, int r) {
        if (!tag[p] || l == r) return;
        int m = l + (r - l) / 2;
        apply(p * 2, l, m, tag[p]);
        apply(p * 2 + 1, m + 1, r, tag[p]);
        tag[p] = 0;
    }
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) { tr[p] = a[l]; return; }
        int m = l + (r - l) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }
    void modify(int p, int l, int r, int L, int R, ll x) {
        if (L <= l && r <= R) { apply(p, l, r, x); return; }
        push(p, l, r);
        int m = l + (r - l) / 2;
        if (L <= m) modify(p * 2, l, m, L, R, x);
        if (R > m) modify(p * 2 + 1, m + 1, r, L, R, x);
        pull(p);
    }
```

DS03 懒标记线段树 Lazy Segment Tree 续 2

接上一页；请合并复制整个模块。源码 data_structure/lazy_segment_tree.hpp

```
    ll query(int p, int l, int r, int L, int R) {
        if (L <= l && r <= R) return tr[p];
        push(p, l, r); // 查询也必须下传，否则子节点仍是旧值。
        int m = l + (r - l) / 2;
        ll ans = 0;
        if (L <= m) ans += query(p * 2, l, m, L, R);
        if (R > m) ans += query(p * 2 + 1, m + 1, r, L, R);
        return ans;
    }
};
```

DS04 单调栈与单调队列 Monotonic

识别：找每个位置左侧最近更小元素；固定窗口查询最小值。

约定：输入和输出均 1 下标；{}或{0}为空。previous_less 无解下标为 0；window_min 返回 ans[l]=min(a[l..l+k-1])，k<=0 或 k>n 返回{0}。

复杂度：均 O(n)时间，每个下标至多进出一次；额外空间 O(n)。

调用：auto left=previous_less(a); auto mn=window_min(a,k); // mn[l]对应窗口左端 l

易错：严格更小必须弹出>=当前值，重复值不能留下。；队列保存下标，才能删除离开窗口的元素。；窗口最小值改最大值时将队尾比较>=改为<=。

例题：a={0,2,2,1,3}的左侧严格更小下标为{0,0,0,0,3}；a={0,4,2,2,5},k=2 的最小值为{0,2,2,2}。实题：洛谷 P1886 滑动窗口，求固定窗口最小/最大值：洛谷 P1886 。

源码 data_structure/monotonic.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 输入 a[1..n]；{}、{0} 均为空数组；返回值也保留 0 号占位。
// 每个位置左侧最近的严格更小值的下标；不存在为 0。
inline vector<int> previous_less(const vector<ll>& a) {
    int n = max(0, int(a.size()) - 1);
    vector<int> ans(n + 1), st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        ans[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    return ans;
}
// 固定长度 k 的窗口最小值；ans[l]=min(a[l..l+k-1])。
// k<=0 或 k>n 返回 {0}。相等时保留较新的下标。
inline vector<ll> window_min(const vector<ll>& a, int k) {
    int n = max(0, int(a.size()) - 1);
    vector<ll> ans(1);
    if (k <= 0 || k > n) return ans;
    deque<int> q;
    for (int i = 1; i <= n; ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

G01 并查集 Disjoint Set Union

识别：只增加连边、判断同集合、维护连通块大小；Kruskal 的基础。

约定：n>=0，点编号 [1,n]；unite 返回是否真的合并；components 是当前块数；size 返回所在块大小。

复杂度：初始化 O(n)，路径减半与按大小合并的操作均摊 O(alpha(n))，空间 O(n)。

调用：cp::DSU d(n); d.unite(u,v); bool ok=d.same(u,v); int cnt=d.size(u);

易错：合并必须先找根；sz 只有根位置有效。；普通 DSU 不支持删边和回滚；多测重新构造。；下标 0 只作预留，不能作为真实点调用。

例题：仓库实用记录：cf/3/1109/3.cpp ，通过偏移边合并后判断置换可达性；已读源码，AC 状态未核验。

源码 graph/dsu.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 点编号 [1,n]；路径减半 + 按集合大小合并。
struct DSU {
    vector<int> p, sz;
    int components;
    explicit DSU(int n) : p(n + 1), sz(n + 1, 1), components(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (p[x] != x) {
            p[x] = p[p[x]];
            x = p[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; --components;
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
} // namespace cp
```

G02 BFS 最短路与路径恢复 BFS Path

识别：无权图最短距离、最少操作次数、输出一条最短路径。

约定：g 长度 n+1，点 [1,n]；边端点必须合法；有向/无向均可，重边自环可用。dist=-1 表示不可达；path(t) 返回起点到 t（含端点），非法或不可达目标返回空。

复杂度：BFS O(n+m)，路径恢复 O(路径长度)，空间 O(n)。

调用：vector<vector<int>> g(n+1); g[u].push_back(v); auto r=cp::bfs(g,s); auto p=r.path(t);

易错：入队立即标记，不能等出队再标记。；仅适用单位边权；0/1 权用 0-1 BFS，非负任意权用 Dijkstra。；无向边必须双向加入；多测传入新图。

例题：来源起点：https://github.com/ball-castle/template/blob/main/bfs.cpp 。新测试中孤立点返回空路径，起点到自身返回单点路径；不声明旧源码 AC。

源码 graph/bfs_path.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// g.size()==n+1；点 [1,n]；有向/无向均可，边权均视为 1。
struct BFSResult {
    vector<int> dist, parent; // 不可达 dist=-1；起点 parent=0
    vector<int> path(int t) const {
        if (t < 1 || t >= (int)dist.size() || dist[t] < 0) return {};
        vector<int> ans;
        for (; t; t = parent[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
inline BFSResult bfs(const vector<vector<int>>& g, int s) {
    int n = (int)g.size() - 1;
    BFSResult r{vector<int>(n + 1, -1), vector<int>(n + 1)};
    if (s < 1 || s > n) return r;
    queue<int> q; q.push(s); r.dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (r.dist[v] == -1) {
            r.dist[v] = r.dist[u] + 1;
            r.parent[v] = u;
            q.push(v);
        }
    }
    return r;
}
} // namespace cp
```

G03 迭代 DFS 连通块与父亲 Iterative DFS Forest

识别：无向图连通块标号、父亲树、需要避开递归栈溢出的深图遍历。

约定：g 长度 n+1，合法点 [1,n]，无向边双向加入；允许自环重边。comp 从 1 开始，根 parent=0，order 是发现顺序。

复杂度：O(n+m) 时间，O(n) 额外空间；显式栈不占递归调用栈。

调用：auto r=cp::dfs_forest(g); int c=r.components; int p=r.parent[u]; bool same=r.comp[u]==r.comp[v];

易错：有向图调用得到的是搜索森林，不是强连通分量。；本模板不输出出栈时间；子树闭区间请用 Euler 模板。；根可能不止一个，处理整张图时不能只从 1 出发。

例题：旧版来源：https://github.com/ball-castle/template/blob/main/dfs.cpp 。新测试覆盖断开的图、自环重边及 200000 点链。

源码 graph/dfs_forest.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向图，点 [1,n]；显式栈模拟 DFS，重边/自环均可。
struct DFSForest {
    vector<int> parent, comp, order;
    int components = 0; // comp 从 1 开始；每个根的 parent=0
};
inline DFSForest dfs_forest(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    DFSForest r;
    r.parent.resize(n + 1); r.comp.resize(n + 1);
    vector<pair<int, int>> st; // (当前点，下一条邻边下标)
    for (int s = 1; s <= n; ++s) if (!r.comp[s]) {
        r.comp[s] = ++r.components;
        r.order.push_back(s); st.push_back({s, 0});
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) { st.pop_back(); continue; }
            int v = g[u][i++];
            if (r.comp[v]) continue;
            r.comp[v] = r.components; r.parent[v] = u;
            r.order.push_back(v); st.push_back({v, 0});
        }
    }
    return r; // order 为入栈顺序，父亲先于儿子
}
} // namespace cp
```

G04 拓扑排序 Topological Sort

识别：DAG 处理顺序、依赖任务、DAG DP、有向环检测。

约定：n>=1，g 长度 n+1，点 [1,n]，有向边；返回全部 n 点的一个合法顺序，有环返回空。允许重边，自环判为环。

复杂度：O(n+m) 时间，O(n) 额外空间。

调用：auto order=cp::topological_sort(g); if(order.empty()) { /* 有环 */ }

易错：重边也要各自计入入度和扣减。；不要把无向边双向加入后直接求拓扑。；普通队列不保证字典序最小；需要时换小根堆。

例题：仓库实例：cf/1704E.cpp，使用拓扑序逆序做 DAG DP；已读源码，AC 状态未核验。

源码 graph/topological_sort.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 有向图，点 [1,n]，n>=1；返回空表示有环；重边计入入度。
inline vector<int> topological_sort(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    vector<int> in(n + 1), order;
    for (int u = 1; u <= n; ++u)
        for (int v : g[u]) ++in[v];
    queue<int> q;
    for (int u = 1; u <= n; ++u) if (!in[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--in[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};
    return order;
}
} // namespace cp
```

G05 简单无向基环图环恢复 Unicyclic Graph

识别：连通简单无向图恰有一个环，找环点、按环顺序做双指针或 DP。

约定：点 [1,n]，每条无向边输入一次；检查 n>=3、边数=n、合法端点、无自环重边与连通性。无效时 ok=false 且 cycle 为空；成功时 cycle 不重复首点，末尾与首点有边。

复杂度：含 set 重边验证 O(n log n) 时间，O(n) 空间；剥叶与环恢复本身 O(n)。

调用：vector<pair<int,int>> edges={{1,2},{2,3},{3,1}}; auto r=cp::unicyclic_graph(3,edges); if(r.ok) for(int u:r.cycle) cout<<u<<' ';

易错：本模板明确拒绝自环、平行边形成的 1/2 长度环。；只有 m=n 不够，还必须连通。；叶子剥除后只剩环点，但仍需恢复相邻顺序。；cycle 下标为 0..长度-1，点编号仍为 1..n。

例题：手算例：边 (1,2),(2,3),(3,1),(3,4)，返回三角环 [1,2,3] 或反向；(1,2) 重复两次的图拒绝。

源码 graph/unicyclic_graph.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct Unicycle {
    bool ok = false;
    vector<int> cycle; // 环顺序，不重复首点；末点与首点相邻
};
// 连通简单无向基环图，点 [1,n]；每条无向边只输入一次。
// 非 n 条边、重边、自环、越界、不连通均返回 {false,{}}。
inline Unicycle unicyclic_graph(int n, const vector<pair<int, int>>& edges) {
    if (n < 3 || (int)edges.size() != n) return {};
    vector<vector<int>> g(n + 1);
    vector<int> deg(n + 1), seen(n + 1);
    set<pair<int, int>> used;
    for (auto [u, v] : edges) {
        if (u < 1 || u > n || v < 1 || v > n || u == v) return {};
        if (u > v) swap(u, v);
        if (!used.insert({u, v}).second) return {};
        g[u].push_back(v); g[v].push_back(u);
        ++deg[u]; ++deg[v];
    }
    queue<int> q; q.push(1); seen[1] = 1;
    int count = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); ++count;
        for (int v : g[u]) if (!seen[v]) seen[v] = 1, q.push(v);
    }
    if (count != n) return {};
    for (int u = 1; u <= n; ++u) if (deg[u] == 1) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); deg[u] = 0;
        for (int v : g[u]) if (deg[v] && --deg[v] == 1) q.push(v);
    }
    int start = 0, length = 0;
    for (int u = 1; u <= n; ++u) if (deg[u]) {
        if (deg[u] != 2) return {};
        if (!start) start = u;
        ++length;
    }
    if (length < 3) return {};
```

G05 简单无向基环图环恢复 Unicyclic Graph 续 2

接上一页；请合并复制整个模块。源码 graph/unicyclic_graph.hpp

```
    Unicycle r; int prev = 0, u = start;
    for (int i = 0; i < length; ++i) {
        r.cycle.push_back(u);
        int next = 0;
        for (int v : g[u]) if (deg[v] && v != prev) { next = v; break; }
        if (!next) return {};
        prev = u; u = next;
    }
    if (u != start) return {};
    r.ok = true;
    return r;
}
} // namespace cp
```

M01 整数与位运算 Integer and Bits

识别：最大公约数、最小公倍数、二进制计数、取最低位、枚举 mask 的所有子集。

约定：gcd_abs 接受完整 ll 范围并返回 unsigned long long；gcd(0,0)=0。lcm_checked 只接受能放进非负 ll 的结果，否则 nullopt。位操作用 ull，位号 0..63；最低位不存在返回-1。

复杂度：gcd/lcm 为 O(log max(|a|,|b|))；64 位基础位操作 O(1)；子集枚举 O(2^popcount(mask))。

调用：ull g=gcd_abs(a,b); auto l=lcm_checked(a,b); if(l) cout<<*l; int cnt=bit_count(mask); int k=lowest_bit_index(mask);

易错：LLONG_MIN 不能直接取负或 abs；其绝对值 2^63 不能装进 ll。；使用 1ULL<<k 且 k<64，不要用有符号 1 处理高位。；子集枚举必须在 sub==0 时结束，否则会回到 mask。

例题：原仓库 gcd.cpp：https://github.com/ball-castle/template/blob/main/gcd.cpp 。旧递归在(-6,-4)返回-2；新版返回非负 gcd 并覆盖完整 64 位输入。

源码 math/integer_bits.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// 绝对值用无符号保存，避免 -LLONG_MIN 的有符号溢出。
inline ull unsigned_abs(ll x) {
    return x < 0 ? 0ULL - ull(x) : ull(x);
}
inline ull gcd_abs(ll a, ll b) {
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    while (y) { ull r = x % y; x = y; y = r; }
    return x;  // gcd(0,0)=0；结果可能为 2^63，不能总转回 ll。
}
inline optional<ll> lcm_checked(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    ull x = unsigned_abs(a), y = unsigned_abs(b);
    __int128 z = (__int128)(x / gcd_abs(a, b)) * y;
    if (z > LLONG_MAX) return nullopt;
    return ll(z);
}

// 位编号从 0 开始；无符号运算可以安全处理第 63 位。
inline ull lowbit(ull x) { return x & (0ULL - x); }
inline int bit_count(ull x) { return popcount(x); }
inline int lowest_bit_index(ull x) {
    return x == 0 ? -1 : int(countr_zero(x));
}
// 第 k 位：1ULL<<k，要求 0<=k<64；不可写 1<<k 处理高位。
// 判断位：(mask>>k)&1ULL；设置位：mask|=(1ULL<<k)。
// 枚举全部子集(含 0)，顺序从大到小，O(2^popcount(mask))：
// for (ull sub=mask;; sub=(sub-1)&mask) {
//     ... 使用 sub ...
//     if (sub==0) break;  // 必须先终止，否则又回到 mask。
// }
```

M02 安全模运算与快速幂 Modular Power

识别：大整数模乘、非负整数快速幂、负数归一化，以及质数条件成立时求逆元。

约定：1<=mod<=LLONG_MAX，底数可为任意 ll；结果在[0,mod)。指数为非负 unsigned long long；0^0 按 1 处理后取模，mod=1 返回 0。

复杂度：归一化/模加/模乘 O(1)，快速幂 O(log exp)。

调用：ll y=mod_pow(a,e,mod); ll p=mod_mul(a,b,mod); 若确认 mod 为质数且 a 不为 0 模 mod，可用 mod_pow(a,mod-2,mod)求逆元。

易错：先转__int128 再乘；乘完再转类型已来不及。；费马逆元 a^(mod-2)只适用于 mod 为质数且 a%mod!=0。；不能把负指数直接转成 unsigned 传入；指数取模也需要额外数论前提。

例题：原仓库 ksm.cpp：https://github.com/ball-castle/template/blob/main/ksm.cpp 。旧(-2)^3 mod5 为-3、3^0 mod1 为 1；新版分别为 2 和 0。

源码 math/modular.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 1<=mod<=LLONG_MAX；返回[0,mod)。支持负数和 LLONG_MIN。
inline ll mod_norm(ll a, ll mod) {
    assert(mod >= 1);
    a %= mod;
    if (a < 0) a += mod;
    return a;
}
inline ll mod_add(ll a, ll b, ll mod) {
    return ll(((__int128)mod_norm(a, mod) + mod_norm(b, mod)) % mod);
}
inline ll mod_mul(ll a, ll b, ll mod) {
    return ll((__int128)mod_norm(a, mod) * mod_norm(b, mod) % mod);
}
// exp 是非负整数；0^0 定义为 1 再取模；模 1 时答案恒为 0。
inline ll mod_pow(ll a, unsigned long long exp, ll mod) {
    a = mod_norm(a, mod);
    ll ans = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = mod_mul(ans, a, mod);
        a = mod_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 费马逆元：仅当 mod 为质数且 a%mod!=0 时，inv=mod_pow(a,mod-2,mod)。
// 复合模数不能套此公式；负指数不能直接传入本函数。
```

DP04 石子合并区间 DP Interval Merge

识别：只能合并相邻部分，最后一次操作把区间分成左右两段，总代价可以由子区间转移。

约定：a 从 1 开始且非负；直线石子合并，单次代价为两堆之和；n=0/1 返回 0。要求 n*sum(a)<2^60；本函数不处理环形合并。

复杂度：时间 O(n^3)，空间 O(n^2)。

调用：vector<ll> a(n+1); 读入 a[1..n]；cout<<interval_merge(a); 该函数求最小代价。

易错：先枚举长度，再枚举左端点和分割点。；前缀和、状态和候选代价均须 64 位；无穷大必须大于合法答案。；环形题通常先复制数组，再取所有长度 n 区间的最优值，不能直接调用直线版。

例题：原仓库区间 dp.cpp：https://github.com/ball-castle/template/blob/main/%E5%8C%BA%E9%97%B4dp.cpp 。两堆各 10^9 的正确答案 2*10^9，旧版返回 1061109567；保留转移并修复整数范围。

源码 dp/interval_merge.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// a 为 1-indexed 非负石子堆。每次只能合并相邻两堆，代价为两堆之和。
// 求全部合成一堆的最小总代价；n=0/1 返回 0；不是环形合并。
// 要求 n*sum(a)<2^60，使前缀和、候选代价及无穷大不混淆。
inline ll interval_merge(const vector<ll>& a) {
    assert(!a.empty());
    int n = int(a.size()) - 1;
    if (n <= 1) return 0;
    vector<ll> sum(n + 1);
    for (int i = 1; i <= n; ++i) {
        assert(a[i] >= 0);
        sum[i] = sum[i - 1] + a[i];
    }
    vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
    // dp[l][r]：合并闭区间[l,r]；先枚举长度，保证子区间已算出。
    for (int len = 2; len <= n; ++len)
        for (int l = 1, r = len; r <= n; ++l, ++r) {
            dp[l][r] = 1LL << 60;
            for (int k = l; k < r; ++k)
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]
                               + sum[r] - sum[l - 1]);
        }
    return dp[1][n];
}
// 时间 O(n^3)，空间 O(n^2)；n 达到千级时应重新评估是否需要优化。
// vector<ll> a(n+1); 读 a[1..n]; cout << interval_merge(a);
```

DS05 稀疏表 Sparse Table

识别：数组不修改，需要大量区间 min/max/gcd 查询。

约定：a[1..n]，{}或{0}为空；非空 query 需 1<=l<=r<=n，空区间返回传入 identity。op 必须结合且幂等；GcdOp 只接收非负 ll。

复杂度：预处理 O(n log n)时间和空间；每次查询 O(1)。

调用：SparseTable<MinOp> st(a,LLONG_MAX); st.query(l,r); SparseTable<GcdOp> sg(diff,0); sg.query(l+1,r);

易错：查询的两块区间可能重叠，不能用于求和。；ST 表只适合静态数组；修改一个元素后不能继续查旧表。；差分 gcd 题查原数组[l,r]时要转为差分[l+1,r]，单点答案为 0。

例题：仓库实例 cf/template/ST 表.cpp 对应 CF2050F：https://codeforces.com/contest/2050/problem/F；相邻差上建 gcd 表，原区间[l,r]转查差分[l+1,r]，单点输出 0。

源码 data_structure/sparse_table.hpp | 测试 tests/test_core_data.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// 静态区间查询；op 必须满足结合律、幂等性，如 min/max/gcd。
// 不支持修改，也不能用两块重叠区间求和。GcdOp 要求输入非负。
struct MinOp {
    ll operator()(ll a, ll b) const { return min(a, b); }
};
struct GcdOp {
    ll operator()(ll a, ll b) const { return gcd(a, b); }
};
template<class Op>
struct SparseTable {
    int n;
    vector<int> lg;
    vector<vector<ll>> st;
    ll identity;
    Op op;
    // a[1..n]，{}、{0} 均为空；empty_value 是空区间返回值。
    SparseTable(const vector<ll>& a, ll empty_value, Op merge = Op{})
        : n(max(0, int(a.size()) - 1)), lg(n + 1),
          identity(empty_value), op(merge) {
        for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vector<ll>(n + 1));
        for (int i = 1; i <= n; ++i) st[0][i] = a[i];
        for (int j = 1; j <= lg[n]; ++j) {
            for (int i = 1; i <= n - (1 << j) + 1; ++i) {
                st[j][i] = op(st[j - 1][i],
                              st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    ll query(int l, int r) const {
        if (l > r) return identity;
        assert(1 <= l && r <= n);
        int k = lg[r - l + 1];
        return op(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

G06 单源与多源最短路 Dijkstra

识别：非负边权最短路；多个起点同时出发时求离最近源点的距离；恢复一条最短路径。

约定：g 长度 n+1，点 [1,n]，边 (v,w)；支持有向、重边、自环、零权。所有边要求 0<=w<DIJK_INF=LLONG_MAX/4；非法边或源点返回 ok=false。空源集合法，全不可达。距离>=INF 饱和为 INF，path 为空。

复杂度：设源数为 s，时间 O(s+(n+m)log(n+m))，额外空间 O(n+m)，使用懒删除堆。

调用：vector<vector<pair<int,ll>>> g(n+1); g[u].push_back({v,w}); auto r=cp::dijkstra(g,vector<int>{s1,s2}); if(r.ok && r.dist[t]!=cp::DIJK_INF) { auto path=r.path(t); } 单源用 cp::dijkstra(g,s)。

易错：不能处理负边，即使负边在不可达分量也会拒绝。；真实有限答案必须严格小于 INF；不能把等于 INF 的答案当可达。；出堆要跳过过期距离，入队不能直接永久标记。；多源路径从实际选中的源点开始，等距时不保证字典序。

例题：手算例：1->2 权 0，2->3 权 5，4->3 权 2；源 {1,4} 时 dist[3]=2，路径 [4,3]。

源码 graph/dijkstra.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
inline constexpr ll DIJK_INF = LLONG_MAX / 4;
struct DijkstraResult {
    bool ok = false;
    vector<ll> dist;
    vector<int> parent;
    vector<int> path(int t) const {
        if (!ok || t < 1 || t >= (int)dist.size()) return {};
        if (dist[t] == DIJK_INF) return {};
        vector<int> ans;
        for (; t; t = parent[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
// 点 [1,n]；g[u] 存 (v,w)。支持有向、重边、零权、多源。
// 拒绝 w<0 或 w>=INF；真实距离>=INF 的点也视为不可达。
```

G06 单源与多源最短路 Dijkstra 续 2

接上一页；请合并复制整个模块。源码 graph/dijkstra.hpp

```
inline DijkstraResult dijkstra(
    const vector<vector<pair<int, ll>>>& g, const vector<int>& sources) {
    int n = (int)g.size() - 1;
    DijkstraResult r{false, vector<ll>(n + 1, DIJK_INF),
                     vector<int>(n + 1)};
    for (int s : sources) if (s < 1 || s > n) return r;
    for (int u = 1; u <= n; ++u) for (auto [v, w] : g[u])
        if (v < 1 || v > n || w < 0 || w >= DIJK_INF) return r;
    r.ok = true;
    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> q;
    for (int s : sources) if (r.dist[s] != 0) {
        r.dist[s] = 0; q.push({0, s});
    }
    while (!q.empty()) {
        auto [du, u] = q.top(); q.pop();
        if (du != r.dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (w > DIJK_INF - du) continue;
            if (du + w < r.dist[v]) {
                r.dist[v] = du + w; r.parent[v] = u;
                q.push({r.dist[v], v});
            }
        }
    }
    return r;
}
inline DijkstraResult dijkstra(
    const vector<vector<pair<int, ll>>>& g, int s) {
    return dijkstra(g, vector<int>{s});
}
} // namespace cp
```

G07 Kruskal 最小生成树与森林 Kruskal MST

识别：无向带权图求最小生成树；图可能不连通时求最小生成森林。

约定：n>=0，合法点 [1,n]；每条无向边输入一次，允许负权、自环、重边。返回 weight、components、connected 和原边数组 0-based 下标 edges；不连通时 weight 是森林总权；n=0 时 connected=false。总权和中间累加必须在 ll 范围内。

复杂度：O(m log m + (n+m)alpha(n)) 时间，O(n+m) 额外空间。

调用：vector<cp::KruskalEdge> e={{1,2,3},{2,3,-1}}; auto r=cp::kruskal(3,e); if(r.connected) cout<<r.weight;

易错：只比较权重，不能将权重强转 int；总权使用 ll。；自环不会被选中，平行边只按最优连接选择。；输出 MST 前先判断 connected；不连通时返回的是森林。；edges 是输入边下标，不是点编号。

例题：旧来源：https://github.com/ball-castle/template/blob/main/kruskal%20MST.cpp，原文件求指定 k 个连通块；新模块提供通常的 MST/森林契约，不冒充原题 AC。

源码 graph/kruskal.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct KruskalEdge { int u, v; ll w; };
struct KruskalResult {
    bool connected = false;
    ll weight = 0; // 不连通时仍返回最小生成森林的总权
    int components = 0;
    vector<int> edges; // 选中边在原输入中的 0-based 下标
};
// 点 [1,n]；无向边只输入一次，支持负权、重边、自环。
// 端点须合法；总权及累加过程须在 ll 范围内。
inline KruskalResult kruskal(int n, const vector<KruskalEdge>& e) {
    vector<int> p(n + 1), sz(n + 1, 1), id(e.size());
    iota(p.begin(), p.end(), 0); iota(id.begin(), id.end(), 0);
    auto find = [&](int x) {
        while (x != p[x]) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    };
    sort(id.begin(), id.end(), [&](int a, int b) {
        return e[a].w < e[b].w;
    });
    KruskalResult r; r.components = n;
    for (int i : id) {
        int a = find(e[i].u), b = find(e[i].v);
        if (a == b) continue;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; --r.components;
        r.weight += e[i].w; r.edges.push_back(i);
    }
    r.connected = (n > 0 && r.components == 1);
    return r;
}
} // namespace cp
```

G08 堆优化 Prim 最小生成森林 Prim MST

识别：已有带权无向邻接表，需要最小生成树或所有分量的最小生成森林。

约定：g 长度 n+1，合法点 [1,n]，无向边双向加入，允许负权、自环、重边。返回 weight、components、connected、parent；每个分量根 parent=0；不连通时仍返回森林权重；n=0 时 connected=false。累加需在 ll 范围内。

复杂度：O((n+m)log(n+m)) 时间，O(n+m) 额外空间；使用边堆，重复候选出堆后忽略。

调用：vector<vector<pair<int,ll>>> g(n+1); g[u].push_back({v,w}); g[v].push_back({u,w}); auto r=cp::prim(g);

易错：选点依据跨越已选/未选集合的最小边，不是到起点的累计路程。；负权可以用 Prim；这一点与 Dijkstra 不同。；无向边必须双向加入；只从 1 遍历不能统计所有分量。；parent 只记录父点，若需具体重边编号可将边 ID 加入堆。

例题：旧来源：https://github.com/ball-castle/template/blob/main/prim%20MST.cpp 。新实现清除多测全局状态依赖，并将总权改为 ll。

源码 graph/prim.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct PrimResult {
    bool connected = false;
    ll weight = 0; // 不连通时为最小生成森林总权
    int components = 0;
    vector<int> parent; // 每个连通块的根为 0
};
// 点 [1,n]；g[u] 存 (v,w)，无向边双向加入；负权/重边可用。
// 端点须合法；累加过程和总权须在 ll 范围内。
inline PrimResult prim(const vector<vector<pair<int, ll>>>& g) {
    int n = (int)g.size() - 1;
    PrimResult r; r.parent.resize(n + 1);
    vector<int> vis(n + 1);
    using State = tuple<ll, int, int>; // 权、点、来自哪个父亲
    priority_queue<State, vector<State>, greater<State>> q;
    for (int s = 1; s <= n; ++s) if (!vis[s]) {
        ++r.components; q.push({0, s, 0});
        while (!q.empty()) {
            auto [w, u, p] = q.top(); q.pop();
            if (vis[u]) continue;
            vis[u] = 1; r.weight += w; r.parent[u] = p;
            for (auto [v, cost] : g[u])
                if (!vis[v]) q.push({cost, v, u});
        }
    }
    r.connected = (n > 0 && r.components == 1);
    return r;
}
} // namespace cp
```

G09 树上倍增最近公共祖先 Binary Lifting LCA

识别：静态树上多次 LCA、两点边数距离、向上跳 k 级祖先查询。

约定：合法点 [1,n]，无向邻接表双向且对称，root 默认 1。检查边数与连通性，非树/非法根时 ok=false。根的父亲为哨兵 0；越界、k<0、跳过根返回 0；无效 lca 返回 0，无效 distance 返回 -1。depth[root]=0。

复杂度：预处理 O(n log n)，查询 O(log n)，空间 O(n log n)；全部迭代，深树栈安全。

调用：cp::LCA tree(g,root); if(tree.ok) { int a=tree.lca(u,v); int p=tree.jump(u,k); int d=tree.distance(u,v); }

易错：先确认 ok，再读取 depth/up 数组。；此实现只用于一棵树；森林需要按分量建树或添加虚根后单独处理跨分量。；点 0 是不存在的祖先，不可作为实际结点。；旧 binary_lifting.cpp 是无法编译的残缺 ST 表，不能当作 LCA。

例题：手算树：1-2,1-3,2-4；根 1 时 lca(4,3)=1，jump(4,2)=1，distance(4,3)=3。

源码 graph/lca.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向树，点 [1,n]，邻接表双向；非法根/非树时 ok=false。
// 邻接端点与双向对称性由调用方保证；根的祖先为哨兵 0。
struct LCA {
    int n, levels = 1;
    bool ok = false;
    vector<int> depth;
    vector<vector<int>> up;
    explicit LCA(const vector<vector<int>>& g, int root = 1)
        : n((int)g.size() - 1) {
        if (n < 1 || root < 1 || root > n) return;
        ll count = 0;
        for (int u = 1; u <= n; ++u) count += (ll)g[u].size();
        if (count != 2LL * (n - 1)) return;
        while ((1LL << levels) <= n) ++levels;
        up.assign(levels, vector<int>(n + 1));
        depth.assign(n + 1, -1); depth[root] = 0;
        vector<int> order{root};
        for (int i = 0; i < (int)order.size(); ++i) {
            int u = order[i];
            for (int v : g[u]) if (depth[v] == -1) {
                depth[v] = depth[u] + 1; up[0][v] = u;
                for (int k = 1; k < levels; ++k)
                    up[k][v] = up[k - 1][up[k - 1][v]];
                order.push_back(v);
            }
        }
        ok = ((int)order.size() == n);
    }
    int jump(int u, ll k) const {
        if (!ok || u < 1 || u > n || k < 0 || k > depth[u]) return 0;
        for (int i = 0; i < levels; ++i) if (k >> i & 1) u = up[i][u];
        return u;
    }
    int lca(int u, int v) const {
        if (!ok || u < 1 || u > n || v < 1 || v > n) return 0;
        if (depth[u] < depth[v]) swap(u, v);
        u = jump(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int k = levels - 1; k >= 0; --k) if (up[k][u] != up[k][v]) {
            u = up[k][u]; v = up[k][v];
        }
        return up[0][u];
    }
```

G09 树上倍增最近公共祖先 Binary Lifting LCA 续 2

接上一页；请合并复制整个模块。源码 graph/lca.hpp

```
    int distance(int u, int v) const {
        int a = lca(u, v);
        return a ? depth[u] + depth[v] - 2 * depth[a] : -1;
    }
};
} // namespace cp
```

G10 Euler 子树区间与迭代树 DP Euler Subtree

识别：将树上子树映射为数组闭区间，配合前缀和/Fenwick/线段树；逆序处理简单树形 DP。

约定：合法点 [1,n]，无向邻接表双向对称；root 默认 1；检查边数和连通性。每点仅计入 Euler 序一次，order[1..n]，order[tin[u]]=u；子树闭区间 [tin[u],tout[u]]。subtree_sum 输入长度 n+1，返回含自身的子树和。无效树或长度不符返回空。

复杂度：预处理、一次子树和 DP 均 O(n)，空间 O(n)，subtree_size 查询 O(1)。

调用：cp::EulerTree e(g,root); if(e.ok) { auto sums=e.subtree_sum(value); int L=e.tin[u], R=e.tout[u]; int size=e.subtree_size(u); }

易错：tout 为子树最后一点的编号，不是第二次访问的时间戳；区间包含两端。；逆序 order 才能先计算儿子，再将状态合并给父亲。；先确认 ok；求和过程须在 ll 范围内。；这不是用于 RMQ 求 LCA 的长度 2n-1 欧拉游走。

例题：仓库树 DP 实例：cf/3/1096/H_%21%21dp_tree.cpp，旧代码递归合并儿子状态；已读源码，AC 未核验。新模板示范同类依赖顺序的子树求和。

源码 graph/euler_subtree.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向树：点 [1,n]，双向邻接，端点合法且对称由调用方保证。
// Euler 序仅入点一次；子树对应闭区间 [tin[u],tout[u]]。
struct EulerTree {
    int n;
    bool ok = false;
    vector<int> tin, tout, parent, order; // order[1..n], order[0]=0
    explicit EulerTree(const vector<vector<int>>& g, int root = 1)
        : n((int)g.size() - 1) {
        if (n < 1 || root < 1 || root > n) return;
        ll count = 0;
        for (int u = 1; u <= n; ++u) count += (ll)g[u].size();
        if (count != 2LL * (n - 1)) return;
        tin.resize(n + 1); tout.resize(n + 1); parent.resize(n + 1);
        vector<pair<int, int>> st{{root, 0}};
        order = {0, root}; tin[root] = 1;
        int timer = 1;
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) {
                tout[u] = timer; st.pop_back(); continue;
            }
            int v = g[u][i++];
            if (tin[v]) continue;
            parent[v] = u; tin[v] = ++timer;
            order.push_back(v); st.push_back({v, 0});
        }
        ok = (timer == n);
    }
    int subtree_size(int u) const {
        return ok && u >= 1 && u <= n ? tout[u] - tin[u] + 1 : 0;
    }
    // 简单树 DP：逆序合并子状态；权值/中间和需在 ll 范围内。
    vector<ll> subtree_sum(const vector<ll>& value) const {
        if (!ok || (int)value.size() != n + 1) return {};
        vector<ll> sum = value;
        for (int i = n; i >= 1; --i) {
            int u = order[i];
            if (parent[u]) sum[parent[u]] += sum[u];
        }
        return sum;
    }
};
} // namespace cp
```

M03 普通模矩阵快速幂 Matrix Power

识别：线性递推阶数很小但 n 很大，能把一次转移写成固定矩阵乘状态列向量。

约定：矩阵 0-indexed，只接受同阶方阵，元素可为任意 ll，1<=mod<=LLONG_MAX；非负 unsigned 指数，0 次幂为单位矩阵。依赖 M02。

复杂度：d 阶矩阵乘法 O(d^3)，幂 O(d^3 log exp)，空间 O(d^2)。

调用：Matrix t={{1,1},{1,0}}; auto r=matrix_pow(t,n,1000000007); cout<<r[0][1]; 工程中头文件自动包含 M02，纸面抄写需带上 M02。

易错：先手算一次转移检查状态顺序，行向量和列向量不能混用。；这是普通模加模乘；最短路 min-plus 和最大值 max-plus 须修改运算及单位元，不能直接用于对应赛题。；负指数不合法；矩阵及中间乘积使用 64 位和__int128。

例题：CSES Fibonacci Numbers：https://cses.fi/problemset/task/1722 。用{{1,1},{1,0}}的 n 次幂，结果[0][1]是 F[n]，包含 n=0；未在线提交。

源码 math/matrix_power.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
#include "modular.hpp"
using namespace std;
using ll = long long;
using Matrix = vector<vector<ll>>;

// 矩阵下标 0-indexed；只接受同阶方阵；1<=mod<=LLONG_MAX。
// 普通模加/模乘，不是最短路 min-plus 或最大值 max-plus 矩阵。
inline Matrix matrix_mul(const Matrix& a, const Matrix& b, ll mod) {
    int n = int(a.size());
    assert(mod >= 1 && b.size() == a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    for (const auto& row : b) assert(int(row.size()) == n);
    Matrix c(n, vector<ll>(n));
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k) {
            ll x = mod_norm(a[i][k], mod);
            if (x == 0) continue;
            for (int j = 0; j < n; ++j) {
                ll y = mod_norm(b[k][j], mod);
                c[i][j] = ll((c[i][j] + (__int128)x * y) % mod);
            }
        }
    return c;
}
inline Matrix matrix_pow(Matrix a, unsigned long long exp, ll mod) {
    assert(mod >= 1);
    int n = int(a.size());
    for (const auto& row : a) assert(int(row.size()) == n);
    Matrix ans(n, vector<ll>(n));
    for (int i = 0; i < n; ++i) ans[i][i] = 1 % mod;
    while (exp) {
        if (exp & 1ULL) ans = matrix_mul(ans, a, mod);
        a = matrix_mul(a, a, mod);
        exp >>= 1;
    }
    return ans;
}
// 例：Matrix t={{1,1},{1,0}}; auto p=matrix_pow(t,n,mod);
// 先写清状态列向量，再验证转移一次；指数 0 返回单位矩阵。
```

M04 乘积递推修正版 Product Recurrence

识别：原 1182E 乘积递推的指数转移示例；用矩阵求各底数指数，再做模幂。理解 M02、M03 后再读。

约定：f[n]=f[n-1]*f[n-2]*f[n-3]*c^(2n-6)，固定模 P=10^9+7；n>=1，四个底数可为任意 ll；n<=3 直接返回初值。依赖 M02、M03。

复杂度：矩阵阶数最多 5，时间 O(5^3 log n)，额外空间 O(5^2)。

调用：ll ans=product_recurrence(n,f1,f2,f3,c); 固定题型示例，不是所有乘积递推通用公式；更换递推时重新推导两个转移矩阵。

易错：累乘答案从 1 开始；原文件从 0 开始会使所有答案为 0。；n>=4 时四个底数的真实指数均为正，先判底数 modP 是否为 0，再把指数模 P-1；不能只凭取模后指数 0 判断 0^0。；原题 n>=4；新版的 n=1/2/3 是明确添加的边界支持，不把原题未要求的输入当作原提交失败证据。

例题：Codeforces 1182E Product Oriented Recurrence：CF 1182/E 。已核对原递推与两条官方样例；不宣称在线 AC。

源码 math/product_recurrence.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
#include "matrix_power.hpp"
using namespace std;
using ll = long long;

// CF1182E: f[n]=f[n-1]*f[n-2]*f[n-3]*c^(2n-6), n>=4。
// n>=1; f1,f2,f3,c 可为任意 ll; 固定模 P=1e9+7。
// 这里额外支持 n=1/2/3 和零底数，原题输入从 n=4 开始。
inline ll product_recurrence(unsigned long long n, ll f1, ll f2,
                             ll f3, ll c) {
    assert(n >= 1);
    const ll p = 1000000007, q = p - 1;
    array<ll, 4> f = {f1, f2, f3, c};
    for (ll& x : f) x = mod_norm(x, p);
    if (n <= 3) return f[n - 1];
    // n>=4 时四个底数的真实指数都为正。先判零，再取指数模 p-1。
    for (ll x : f) if (x == 0) return 0;
    Matrix t = {{1, 1, 1}, {1, 0, 0}, {0, 1, 0}};
    Matrix a = matrix_pow(t, n - 3, q);
    // c 的指数 E: E[t+1]=E[t]+E[t-1]+E[t-2]+2*t-4。
    // 状态列向量为 [E[t],E[t-1],E[t-2],t,1]，初态[0,0,0,3,1]。
    Matrix u = {
        {1, 1, 1, 2, -4}, {1, 0, 0, 0, 0},
        {0, 1, 0, 0, 0}, {0, 0, 0, 1, 1}, {0, 0, 0, 0, 1}
    };
    Matrix b = matrix_pow(u, n - 3, q);
    ll ec = mod_add(mod_mul(b[0][3], 3, q), b[0][4], q);
    array<ll, 4> e = {a[0][2], a[0][1], a[0][0], ec};
    ll ans = 1;  // 连乘单位元是 1，不能从 0 开始。
    for (int i = 0; i < 4; ++i)
        ans = mod_mul(ans, mod_pow(f[i], e[i], p), p);
    return ans;
}
```

M05 质数模数组合数 Prime Combinations

识别：大量询问 C(n,k)，所有 n 有可承受的预处理上限，模数已知为质数且大于该上限。

约定：prime 必须为质数且 0<=max_n<prime；模板不检测质数。有效查询须 0<=n<=max_n；k<0 或 k>n 返回 0。依赖 M02。

复杂度：预处理 O(max_n+log prime) 时间、O(max_n) 空间；每次查询 O(1)。

调用：PrimeComb comb(max_n,1000000007); cout<<comb.choose(n,k); 纸面抄写同时带上 M02。

易错：阶乘含模数因子时没有逆元：n>=prime 不能继续套此模板。；复合模数不能使用费马逆元；多测可按全体最大 n 预处理一次。；C(0,0)=1，查询上限不能大于构造时的 max_n。

例题：Codeforces 1475E Advertising Agency：CF 1475/E 。对选取边界的相同值计算组合数；未在线提交。

源码 math/combinations.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
#include "modular.hpp"
using namespace std;
using ll = long long;

// 仅适用于已知质数 prime，并且 0<=预处理上限 n<prime。
// 本模板不检测质数；复合模数或 n>=prime 不能用阶乘逆元法。
struct PrimeComb {
    ll mod;
    vector<ll> fac, ifac;
    PrimeComb(int n, ll prime) : mod(prime) {
        assert(n >= 0 && prime >= 2 && ll(n) < prime);
        fac.resize(n + 1); ifac.resize(n + 1);
        fac[0] = 1;
        for (int i = 1; i <= n; ++i)
            fac[i] = mod_mul(fac[i - 1], i, mod);
        ifac[n] = mod_pow(fac[n], mod - 2, mod);
        for (int i = n; i >= 1; --i)
            ifac[i - 1] = mod_mul(ifac[i], i, mod);
    }
    ll choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        assert(n >= 0 && n < int(fac.size()));
        return mod_mul(fac[n], mod_mul(ifac[k], ifac[n - k], mod), mod);
    }
};
// PrimeComb comb(max_n,1000000007); cout << comb.choose(n,k);
// C(0,0)=1；无效 k 返回 0；有效查询的 n 不能超过预处理上限。
// 预处理 O(n+log mod)时间/O(n)空间，每次查询 O(1)。
```

S01 KMP 与 Z 函数 Prefix Matching

识别：线性时间寻找全部模式串出现位置、分析前后缀相等长度，或比较字符串每个后缀与全串前缀。

约定：字符串按字节、0-indexed，长度能放进 int。KMP 返回所有匹配起点，允许重叠；空模式匹配 0..n。pi 为最长相等真前后缀长度；Z 约定 z[0]=0。

复杂度：KMP O(|s|+|p|) 时间，O(|p|+匹配数)空间；pi/Z 为 O(n) 时间和空间。

调用：auto pos=kmp_find_all(text,pattern); auto pi=prefix_function(s); auto z=z_function(s); 字符串无需人为补第 0 个字符。

易错：完全匹配后令 j=pi[j-1]，否则会漏重叠匹配或越界。；不同资料可能定义 z[0]=n，使用公式时先核对约定。；空模式不能直接访问 pi[-1]；新版先单独处理。

例题：CSES String Matching：https://cses.fi/problemset/task/1753 。调用 kmp_find_all(text,pattern).size()；函数来自原 KMP.cpp 的转移提炼并修复空模式边界，未在线提交。

源码 string/kmp_z.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 字符串统一 0-indexed。pi[i]是 s[0..i]最长相等真前后缀的长度。
inline vector<int> prefix_function(const string& s) {
    vector<int> pi(s.size());
    for (int i = 1; i < int(s.size()); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
// 返回 p 在 s 中所有起点，允许重叠；空 p 匹配 0..s.size()共 n+1 处。
inline vector<int> kmp_find_all(const string& s, const string& p) {
    vector<int> ans;
    if (p.empty()) {
        ans.resize(s.size() + 1); iota(ans.begin(), ans.end(), 0);
        return ans;
    }
    auto pi = prefix_function(p);
    for (int i = 0, j = 0; i < int(s.size()); ++i) {
        while (j && s[i] != p[j]) j = pi[j - 1];
        if (s[i] == p[j]) ++j;
        if (j == int(p.size())) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}
// z[i]是 s 与 s[i..]的最长公共前缀长度；本实现约定 z[0]=0。
inline vector<int> z_function(const string& s) {
    int n = int(s.size());
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        // [l,r) 是已知最靠右的匹配区间。
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
// kmp_find_all(text,pattern)无需选分隔符，适合直接查找。
```

S02 双模字符串哈希 Double Hash

识别：大量查询子串指纹、子串去重或快速筛选相等候选；可以接受碰撞风险时使用。

约定：字符串 0-indexed，get(l,r)取半开区间[l,r)，0<=l<=r<=n；长度能放进 int。按 unsigned char+1 编码，固定 base 与两组模数。比较哈希时也比较长度。

复杂度：预处理 O(n) 时间、O(n) 空间；子串哈希 O(1)，确定性字符串核验另计。

调用：DoubleHash h(s); auto key=h.get(l,r); 若比较区间先确认长度相等，再比较 key；空区间的 key 为(0,0)。

易错：双模仍有碰撞，固定 base 还可能被针对构造；随机测试不能证明无碰撞。；需要确定性匹配可用 KMP；重要哈希命中要按原串核验。；闭区间[l,r]调用时右端改为 r+1；不同哈希参数的对象不能互相比。

例题：Codeforces 271D Good Substrings：CF 271/D 。可用坏字符前缀和筛选，再对(长度,双哈希)去重；这仍是有碰撞风险的方案，未在线提交。

源码 string/double_hash.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 0-indexed 字符串；get(l,r)取半开区间[l,r)，要求 0<=l<=r<=n。
// 双模哈希仍可能碰撞，不能作为无碰撞证明；重要比较须直接核验。
// 比较前同时比较长度；不同对象使用相同 base 和模数才可比较。
struct DoubleHash {
    static constexpr ll mod1 = 1000000007, mod2 = 1000000009;
    static constexpr ll base = 911382323;
    vector<ll> h1, h2, p1, p2;
    explicit DoubleHash(const string& s) {
        int n = int(s.size());
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            // unsigned char 避免高位字节被解释成负数；+1 不把零字节当 0。
            ll x = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * base + x) % mod1;
            h2[i + 1] = (h2[i] * base + x) % mod2;
            p1[i + 1] = p1[i] * base % mod1;
            p2[i + 1] = p2[i] * base % mod2;
        }
    }
    pair<ll, ll> get(int l, int r) const {
        assert(0 <= l && l <= r && r < int(h1.size()));
        ll a = (h1[r] - h1[l] * p1[r - l] % mod1 + mod1) % mod1;
        ll b = (h2[r] - h2[l] * p2[r - l] % mod2 + mod2) % mod2;
        return {a, b};
    }
};
// DoubleHash h(s); auto key=h.get(l,r); 空区间返回(0,0)。
// 固定 base 方便复现，但可被针对构造；需要确定性匹配优先用 KMP。
```

S03 基础字符串处理 String Basics

识别：回文检查、单字符分割、ASCII 大小写、相邻相同字符分组，以及含空格输入。

约定：按字节处理而非 Unicode 字符，长度能放进 int；回文使用 0-indexed 半开区间[l,r)。分割保留所有空字段；大小写转换只处理 ASCII 大写字母。

复杂度：各函数线性时间；回文 O(1)额外空间，其余按返回内容 O(n)空间。

调用：bool ok=is_palindrome(s,0,int(s.size())); auto fields=split_char(s,','); auto groups=run_length(s); string lower=ascii_lower(s);

易错：cin>>后接 getline 要处理残留换行；cin>>ws 会吞掉有意义的前导空白。；split_char(空串)返回一个空字段；末尾分隔符也产生空字段。；UTF-8 中文由多个字节构成，不能把逐字节操作当成逐汉字处理。

例题：Codeforces 266A Stones on the Table：CF 266/A 。相邻同色分组后累加每组长度减 1；未在线提交。

源码 string/basic_string.hpp | 测试 tests/test_dp_math_string.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 基础字符串均按字节处理；0-indexed，区间[l,r)，不是 Unicode 码点。
inline bool is_palindrome(const string& s, int l, int r) {
    assert(0 <= l && l <= r && r <= int(s.size()));
    for (--r; l < r; ++l, --r) if (s[l] != s[r]) return false;
    return true;
}
// 按单个字符分割，保留首尾及连续分隔符产生的空串。
inline vector<string> split_char(const string& s, char sep) {
    vector<string> parts;
    size_t start = 0;
    for (size_t i = 0; i <= s.size(); ++i)
        if (i == s.size() || s[i] == sep) {
            parts.push_back(s.substr(start, i - start));
            start = i + 1;
        }
    return parts;
}
inline string ascii_lower(string s) {
    for (char& c : s) if ('A' <= c && c <= 'Z') c = char(c - 'A' + 'a');
    return s;
}
inline vector<pair<char, int>> run_length(const string& s) {
    vector<pair<char, int>> groups;
    for (char c : s) {
        if (groups.empty() || groups.back().first != c)
            groups.push_back({c, 1});
        else ++groups.back().second;
    }
    return groups;
}
// 读含空格整行：getline(cin,s)；刚用 cin>>后先丢弃残留换行。
// cin.ignore(numeric_limits<streamsize>::max(),'\n');
// cin>>ws 会跳过全部前导空白；若空白有意义，不要使用它。
```

G11 强连通分量 Iterative Kosaraju SCC

识别：有向图中两两互相可达的分量；缩点成 DAG 的第一步。

约定：g 长度 n+1，合法点 [1,n]；允许重边自环。返回 count、comp[1..n]、groups[1..count]；0 为保留位置。

复杂度：O(n+m) 时间和空间；两次显式栈遍历，额外存反图。

调用：auto r=cp::scc(g); bool mutual=r.comp[u]==r.comp[v]; for(int u:r.groups[c]) { /* 分量内结点 */ }

易错：第一遍要记录退出顺序，不能使用刚发现结点的顺序。；第二遍使用反图并按第一遍退出顺序的逆序搜索。；缩点时需另外遍历原边，过滤同分量边；是否去重依题意。

例题：手算例：1->2,2->1,2->3,3->3；分量为 {1,2} 和 {3}，缩点后只有一条跨分量边。

源码 advanced/scc.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
struct SCCResult {
    int count = 0;
    vector<int> comp;
    vector<vector<int>> groups{{}}; // groups[1..count]
};
// Kosaraju 两遍迭代 DFS；有向图，合法点 [1,n]，可有重边自环。
inline SCCResult scc(const vector<vector<int>>& g) {
    int n = (int)g.size() - 1;
    vector<vector<int>> rev(n + 1);
    for (int u = 1; u <= n; ++u)
        for (int v : g[u]) rev[v].push_back(u);
    vector<int> seen(n + 1), finish;
    vector<pair<int, int>> st;
    for (int s = 1; s <= n; ++s) if (!seen[s]) {
        seen[s] = 1; st.push_back({s, 0});
        while (!st.empty()) {
            auto& [u, i] = st.back();
            if (i == (int)g[u].size()) {
                finish.push_back(u); st.pop_back(); continue;
            }
            int v = g[u][i++];
            if (!seen[v]) { seen[v] = 1; st.push_back({v, 0}); }
        }
    }
    SCCResult r; r.comp.resize(n + 1);
    vector<int> stack;
    reverse(finish.begin(), finish.end());
    for (int s : finish) if (!r.comp[s]) {
        ++r.count; r.groups.push_back({});
        r.comp[s] = r.count; stack.push_back(s);
        while (!stack.empty()) {
            int u = stack.back(); stack.pop_back();
            r.groups.back().push_back(u);
            for (int v : rev[u]) if (!r.comp[v]) {
                r.comp[v] = r.count; stack.push_back(v);
            }
        }
    }
    return r;
}
} // namespace cp
```

G12 无向图找桥 Iterative Bridges

识别：找删除后连通块数增加的边；边双连通分量与桥树的基础。

约定：n>=0，合法点 [1,n]；无向边每条输入一次，允许重边、自环和不连通图。返回桥在原 edges 中的 0-based 下标，按输入顺序排列。

复杂度：O(n+m) 时间与空间；迭代 low-link，不占递归调用栈。

调用：vector<pair<int,int>> e={{1,2},{1,2},{2,3},{3,3}}; auto ids=cp::bridges(3,e);

易错：必须按边 ID 跳过唯一父边；直接跳过父点会把平行边误判为桥。；判断桥使用 low[child] > tin[parent]，不是 >=。；退栈后才把儿子的 low 合并到父亲；自环永远不是桥。

例题：手算例：两条平行边 (1,2)、一条 (2,3)、一条自环 (3,3)，只有原输入下标 2 的 (2,3) 是桥。

源码 advanced/bridges.hpp | 测试 tests/test_graph.cpp | 独立编译及所属套件通过；ASan/UBSan 已运行

```
#pragma once
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

namespace cp {
// 无向图，合法点 [1,n]；每条边输入一次，允许重边和自环。
// 返回桥在原 edges 中的 0-based 下标，按输入顺序排列。
inline vector<int> bridges(int n, const vector<pair<int, int>>& edges) {
    int m = (int)edges.size();
    vector<vector<pair<int, int>>> g(n + 1);
    for (int i = 0; i < m; ++i) {
        auto [u, v] = edges[i];
        g[u].push_back({v, i}); g[v].push_back({u, i});
    }
    vector<int> tin(n + 1), low(n + 1), parent(n + 1);
    vector<int> pe(n + 1, -1), next(n + 1), mark(m), st;
    int timer = 0;
    for (int s = 1; s <= n; ++s) if (!tin[s]) {
        tin[s] = low[s] = ++timer; st.push_back(s);
        while (!st.empty()) {
            int u = st.back();
            if (next[u] < (int)g[u].size()) {
                auto [v, id] = g[u][next[u]++];
                if (id == pe[u]) continue; // 只跳过同一条父边
                if (!tin[v]) {
                    parent[v] = u; pe[v] = id;
                    tin[v] = low[v] = ++timer; st.push_back(v);
                } else low[u] = min(low[u], tin[v]);
            } else {
                st.pop_back(); int p = parent[u];
                if (p) {
                    if (low[u] > tin[p]) mark[pe[u]] = 1;
                    low[p] = min(low[p], low[u]);
                }
            }
        }
    }
    vector<int> ans;
    for (int i = 0; i < m; ++i) if (mark[i]) ans.push_back(i);
    return ans;
}
} // namespace cp
```